matrices," *SIAM J. Appl. Math.*, vol. 12, pp. 515-522, Sept. 1964.
[3] S. Zohar, "The solution of a Toeplitz set of linear equations," *J. Ass. Comput. Mach.*, vol. 21, pp. 272-276, Apr. 1974.
[4] E. H. Bareiss, "Numerical solution of linear equations with Toeplitz and vector Toeplitz matrices," *Numer. Math.*, vol. 13, pp. 404-424, Oct. 1969.
[5] T. Kailath, A. Vieira, and M. Morf, "Inverses of Toeplitz operators, innovations, and orthogonal polynomials," presented at IEEE Conf. Decision and Control, Hyatt Regency Houston, Houston, TX, Dec. 10-12, 1975.
[6] G. Szegö, *Orthogonal Polynomials*, vol. 23, 3rd ed. (Amer. Math. Soc.). New York: Colloquium, 1967.
[7] G. Baxter, "Polynomials defined by a difference system," *J. Math. Anal. Appl.*, vol. 2, pp. 223-263, Apr. 1961.
[8] E. O. Brigham, *The Fast Fourier Transform*. Englewood Cliffs, NJ: Prentice-Hall, 1974.
[9] A. K. Jain, "Fast inversion of banded Toeplitz matrices by circular decomposition," *IEEE Trans. Acoust., Speech, Signal Processing*, vol. ASSP-26, pp. 121-126, Apr. 1978.
[10] S. Zohar, "Toeplitz matrix inversion: The algorithm of W. F. Trench," *J. Ass. Comput. Mach.*, vol. 16, pp. 592-601, Oct. 1969.
[11] J. H. Justice, "The Szegö recursion relation and inverses of positive definite Toeplitz matrices," *SIAM J. Math. Anal.*, vol. 5, pp. 503-508, May 1974.

A Simple Fixed-Point Error Bound for the Fast Fourier Transform

WILLIAM R. KNIGHT AND R. KAISER

Abstract—**Error bounds for the computation of the fast Fourier transform in fixed-point arithmetic are derived for any arithmetic number base and for any prime factorization of the data array length. The intended application is for signal processing with minicomputers. Errors arising from inaccurate sine coefficients and from limited arithmetic precision are considered. The arithmetic error depends essentially on shifts of the data array that may be required to avoid overflow of the computer word. Our closest bound requires knowledge of where shifts occur and is best computed in parallel with the Fourier transform. For the case that such program modification is not feasible, we derive an error bound for *a posteriori* calculation and an *a priori* error estimate. Our bounds are for the maximum error because little is gained at the expense of considerably greater complexity for probabilistic error bounds.**

I. Introduction

An increased awareness of the limitations of fixed-point arithmetic has come with the growing use of micro- and minicomputers for data acquisition and processing in physical and chemical instrumentation. The work reported here arose from an analysis of observations [1] in Fourier transform nuclear magnetic resonance spectroscopy where the fast Fourier transform algorithm is commonly implemented with fixed-point arithmetic. The limitation imposed by the finite computer word length becomes evident in the form of a restricted "dynamic range" and as computational "noise." These effects are caused by the need to truncate the product or to shift (divide by 2) the sum of two fixed-point numbers in order to fit the result into the computer word.

The error so arising from computing the fast Fourier transform (FFT) in fixed-point arithmetic has been analyzed previously. Welch [2] has analyzed the case characterized by rounded sign-magnitude binary arithmetic using a floating-block decimation-in-time radix-2 FFT algorithm and assuming data such that successive stages of computation are statistically independent. Oppenheim and Weinstein [3] analyze the case characterized by rounded binary arithmetic (sign-magnitude format seems to be assumed in some places) using a decimation-in-time radix-2 FFT algorithm with white noise as data causing either no shift or a shift at each stage. Tran-Thong and Liu [4] have treated all cases generated by rounded or truncated 2-complement binary arithmetic using a floating-block decimation-in-time, or frequency, radix-2 FFT algorithm with data making successive stages statistically independent and requiring either no shift in any stage or one shift in each stage.

Our analysis follows Welch [2] in spirit but covers truncated, or rounded, complement or sign-magnitude arithmetic to any number base using floating-block decimation-in-time, or frequency, mixed-radix FFT algorithms with data requiring any possible number of scaling shifts. Following the tradition of Wilkinson [5], our bounds are worst case rather than probabilistic, thus avoiding the assumption that errors are independent or else the complications and uncertainties arising from correlation of error and signal. Surprisingly, the worst case differs from the probable error only by a multiplicative constant rather than by a factor proportional to the square root

Manuscript received August 23, 1978; revised July 1, 1979.

W. R. Knight is with the Departments of Computer Science and Mathematics, University of New Brunswick, Fredericton, N.B., Canada.
R. Kaiser is with the Department of Physics, University of New Brunswick, Fredericton, N.B., Canada.

0096-3518/79/1200-0615$00.75

of the number of stages in the computation. On the other hand, statistical assumptions can assure us that the error is distributed uniformly over the elements of the output data vector while our worst case analysis does not prevent it from being concentrated in a single element.

In addition to the arithmetic error, we consider a "trigonometric error" that arises from lack of precision in the sine and cosine coefficients needed for the Fourier transform. These values can either be calculated from a series approximation to the sine function [6], or they can be stored in a table. In the first case, the error arises from both the computer arithmetic and from truncation of the series. In the second case, interpolation between tabulated values becomes necessary for sizeable data arrays occurring in practice. Interpolation can be linear [7], in which case most of the trigonometric error arises from the chord approximation to the sine curve, or by means of the nonlinear sin $(\alpha+\beta)$ addition theorem [8], in which case it arises from the interpolation arithmetic.

For binary arithmetic and for a complex data vector of length $N=2^n$, our results can be summarized as follows. We let x_o be the vector carrying the input data whose Fourier transform is to be computed. The fast Fourier transform algorithm proceeds through n stages, successively replacing the input vector x_o by $x_1, x_2, \cdots, x_n$, the last vector being the desired transform

$$x_n(f)=\sum_{t=0}^{N-1} x_o(t)\exp(-2\pi ift/N),$$

except for scaling as needed to avoid overflow of the computer word and except for computation noise. The total noise is bounded by the sum of the trigonometric noise plus the arithmetic noise.

The trigonometric noise/signal ratio is bounded by $2\sqrt{2}n\epsilon$, where ϵ bounds the absolute error in the sine and cosine values.

The arithmetic noise/signal ratio is bounded by $r_n u/\|x_o\|$, where u is the value of the least positive representable number in units of the input signal x_o, and the double bars represent the norm which we take as the rms value $\|x\|=(\sum_{q=0}^{N-1}|x(q)|^2/N)^{1/2}$. The normalized arithmetic error r_n is bounded by the sum $r_n<3.81\ \sum_{k=1}^{n} 2^{s_k-(k/2)}$. The constant 3.81 is derived for truncating arithmetic; a smaller value holds for rounding arithmetic. The quantity 2^{s_k} is the scale factor at the end of the kth stage, i.e., s_k is the number of scaling right shifts accumulated to the end of the kth stage as required to avoid overflow of the computer word.

The computation of r_n can easily be included in the FFT program, accumulating the kth summand at the kth stage of computation. However, failing this, we derive the upper bound $r_n<26.4\times 2^{(sn+1)/2}$.

Our bound on the trigonometric noise/signal ratio is *a priori*; it can be evaluated before computation starts. However, our bound on the arithmetic noise/signal ratio is *a posteriori* because its evaluation requires knowledge of the scaling shifts, which knowledge becomes available only as computation proceeds. Since an *a priori* bound for the noise/signal ratio is highly desirable,[1] we can combine the last expression above with an estimate of the total number of required shifts $s_n\simeq(n/2)+\log_2(\rho_n/\rho_o)\leqslant n+1$. In this expression, ρ_o and ρ_n are the peak/rms ratios of the input and output signals x_o and x_n, respectively. Strictly, the spectrum ratio ρ_n is known *a posteriori* only, but sufficient information about the signal spectrum will often be available to permit a reasonable *a priori* estimate.

The following Sections II and III show the derivation of the bounds on the arithmetic noise/signal ratio for any base b of computer arithmetic and for algorithms of mixed radix. Derivation of the error bound for individual arithmetic operations is delegated to Appendix I. The trigonometric error is dealt with in Section IV, and Section V gives a brief outline of difficulties with statistical error analysis.

When the input data are real rather than complex, the FFT algorithm may be adapted to take advantage of the redundancy that arises from the vanishing imaginary part of the input vector. Appendix II shows that our error analysis adapts easily to this case.

II. Arithmetic Error and General Considerations

The recursion leading from one stage to the next in the FFT algorithm is

$$x_k=F_k x_{k-1} \tag{1}$$

where F_k is an $N\times N$ matrix of simple structure [9], [10]. Although the details differ for different versions of the algorithm, the F_k are unitary matrices for theoretical discussions. For practical computations, complex numbers are represented by real and imaginary parts. Complex vectors of length N then become real vectors of length $2N$, and the F_k become $2N\times 2N$ real orthogonal matrices. For fixed-point computations, considerations of scaling and the desire for simple multipliers intervene, and each F_k is some multiple of an orthogonal $2N\times 2N$ matrix. Instead of Parseval's theorem, we then have, at any stage k

$$\|x_k\|=\|F_k\|\cdot\|F_{k-1}\|\cdot\cdots\cdot\|F_1\|\cdot\|x_o\|, \tag{2}$$

where $\|F\|$ indicates the spectral norm [11] of the matrix F, $\|F\|=\max_{\|x\|=1}\|Fx\|$.

In finite precision arithmetic, the matrix $\times$ vector product will generate an error vector, say d_k, and the computed recursion really is

$$\tilde{x}_k=F_k\tilde{x}_{k-1}+d_k. \tag{3}$$

Beginning with $\tilde{x}_1$, the computed data vectors $\tilde{x}_k$ will differ from (1) by an error vector e_k which builds up according to the recursion

$$e_k=F_k e_{k-1}+d_k;\qquad e_o=0. \tag{4}$$

By taking norms on both sides, we get by the triangle inequality

$$\|e_k\|\leqslant\|F_k\|\cdot\|e_{k-1}\|+\|d_k\|;\qquad \|e_o\|=0 \tag{5}$$

[1]We gratefully acknowledge the comments of a referee who impressed upon us the desirability of an *a priori* error estimate.

which, after division by the norm of (1), gives a bound for the propagation of the noise/signal ratio $R_k = \|e_k\|/\|x_k\|$

$$R_k \leqslant R_{k-1} + \|d_k\|/\|x_k\|; \quad R_o = 0. \tag{6}$$

The recursion (6) establishes our basic bound on the arithmetic noise/signal ratio. Evaluation of this recursion depends on particular features of the computation and is the subject of the next section.

III. Arithmetic Error and Particular Considerations

The kth stage deals with the prime factor p_k in the factorization

$$N = p_1 p_2 \cdots p_n. \tag{7}$$

In the complex domain, each element of the new vector x_k is the weighted sum of p_k elements of the old vector x_{k-1}, the weights being the "twiddle factors" $\exp(i\theta)$. The algorithm represents complex numbers by their real and imaginary parts, and the $2N \times 2N$ matrix F_k has, therefore, p_k sines and p_k cosines of p_k angles θ in each row, all other elements being zero. The spectral norm is $\|F_k\| = \sqrt{p_k}$, i.e., the rms value of the data vector $\|x_k\|$ increases by a factor $\sqrt{p_k}$ in the kth stage.

This increase may make it necessary to shift the data array one (or more) computer digit, i.e., to divide by the number base b of the computer arithmetic in order to avoid overflow of the computer word. We let s_k be the number of shifts accumulated up to the end of the kth stage. Incorporation of the shift into the matrix F_k gives then $\|F_k\| = b^{-(s_k - s_{k-1})}\sqrt{p_k}$. Accumulation of these scale factors in (2) gives us, for the rms value of the data array at the end of the kth stage,

$$\|x_k\| = b^{-s_k}\sqrt{N_k}\,\|x_o\| \tag{8}$$

with the abbreviation $N_k = p_1 p_2 \cdots p_k$.

The elements of the error vector d_k will be small multiples of the least positive number representable in the computer word format. We derive in Appendix I an upper bound c_k for this multiple at stage k, and we let u be the value of the least positive computer number in units of the input data x_o, so that $\|d_k\| \leqslant c_k u$. The recursion (6) thereby becomes

$$R_k \leqslant R_{k-1} + c_k b^{s_k} u/(\|x_o\|\sqrt{N_k}); \quad R_o = 0. \tag{9}$$

In terms of the normalized noise/signal ratio $r_k = R_k\|x_o\|/u$, it evaluates to

$$r_n \leqslant \sum_{k=1}^{n} c_k b^{s_k}/\sqrt{N_k}. \tag{10}$$

This is our basic bound on the arithmetic noise/signal ratio which was summarized in Section I for binary arithmetic and $N = 2^n$.

In order to evaluate (10) other than in parallel with the computation of the Fourier transform, we need to bound $b^{s_k}/\sqrt{N_k}$. There are, in fact, two bounds available. The first follows simply from the fact that s_k, the number of shifts at the end of the kth stage, cannot exceed s_n, the total number of shifts at the end of the computation, thus

$$b^{s_k}/\sqrt{N_k} \leqslant b^{s_n}/\sqrt{N_k}. \tag{11}$$

To obtain the second bound, we separate the real and imaginary parts of the vectors x_k and treat each x_k as a real $2N$ vector. Each element of x_k is a weighted and scaled sum of $2N_k$ elements of x_o, and the sum of the absolute values of the weights is, at most, $N_k\sqrt{2}$. We consider ξ_k, the largest absolute value of any element of x_k, and obtain

$$\xi_k \leqslant \xi_o N_k b^{-s_k}\sqrt{2}. \tag{12}$$

Moreover, to make full use of the computer word, ξ_k/ξ_o must be held within the limits

$$b^{-1} \leqslant \xi_k/\xi_o \leqslant b. \tag{13}$$

Substitution from (12) gives the second bound

$$b^{s_k}/\sqrt{N_k} \leqslant b\sqrt{2N_k}. \tag{14}$$

Bound (14) increases with increasing N_k, while bound (11) decreases. The two bounds are equal for

$$N_* = b^{s_n - 1}/\sqrt{2}, \tag{15}$$

and we let m be the least positive integer for which $N_m \geqslant N_*$. For $k < m$, we use bound (14) and get

$$b^{s_k}/\sqrt{N_k} \leqslant b\sqrt{2N_k} = b^{(s_n+1)/2}\,2^{1/4}\sqrt{N_k/N_*}; \quad k < m. \tag{16a}$$

For $k \geqslant m$, we use bound (11) and get

$$b^{s_k}/\sqrt{N_k} = b^{s_k-(s_n-1)/2}\,2^{1/4}\sqrt{N_*/N_k} \leqslant b^{(s_n+1)/2}\,2^{1/4} \cdot\sqrt{N_*/N_k}; \quad k \geqslant m. \tag{16b}$$

We are now in a position to bound the sum (10). With c being the largest of the c_k, application of (16a) and (16b) gives

$$r_n \leqslant c\,2^{1/4}\,b^{(s_n+1)/2}\left(\sqrt{\frac{N_1}{N_*}} + \sqrt{\frac{N_2}{N_*}} + \cdots + \sqrt{\frac{N_{m-1}}{N_*}} + \sqrt{\frac{N_*}{N_m}} + \cdots + \sqrt{\frac{N_*}{N_n}}\right). \tag{17}$$

Formula (17) is convex in $\sqrt{N_*}$ and takes its maximum for either $N_* = N_{m-1}$ or for $N_* = N_m$ at the extremes of the range of N_*. In the first case, the parentheses in (17) has the maximum value

$$\left(\sqrt{\frac{N_1}{N_{m-1}}} + \cdots + \sqrt{\frac{N_{m-2}}{N_{m-1}}} + 1 + \sqrt{\frac{N_{m-1}}{N_m}} + \cdots + \sqrt{\frac{N_{m-1}}{N_n}}\right) \tag{18a}$$

and in the second case its maximum is

$$\left(\sqrt{\frac{N_1}{N_m}} + \cdots + \sqrt{\frac{N_{m-1}}{N_m}} + 1 + \sqrt{\frac{N_m}{N_{m+1}}} + \cdots + \sqrt{\frac{N_m}{N_n}}\right). \tag{18b}$$

Since 2 is the smallest prime, the square roots are maximized as

$$\sqrt{N_k/N_{k+l}} \leqslant 2^{-l/2}$$

so that both (18a) and (18b) are bounded by

$$(\cdots + 2^{-3/2} + 2^{-2/2} + 2^{-1/2} + 1 + 2^{-1/2} + 2^{-2/2} + 2^{-3/2} + \cdots). \tag{18c}$$

The series with quotients $2^{-1/2}$ can be summed to infinity in both directions, and the result is

$$r_n < c2^{1/4} \frac{\sqrt{2}+1}{\sqrt{2}-1} b^{(s_n+1)/2}. \tag{19}$$

This is the *a posteriori* bound which was summarized in Section I for binary arithmetic.

It remains to obtain an *a priori* estimate of the total number of scaling shifts s_n. If, for comparing peak/rms ratios ρ in the input and output data vectors, we divide (13) by (8) with $k = n$, the result is

$$\frac{b^{s_n-1}}{\sqrt{N}} \leqslant \frac{\rho_n}{\rho_o} \leqslant \frac{b^{s_n+1}}{\sqrt{N}} \tag{20}$$

and hence

$$\log_b\left(\frac{\rho_n}{\rho_o}\sqrt{N}\right) - 1 \leqslant s_n \leqslant \log_b\left(\frac{\rho_n}{\rho_o}\sqrt{N}\right) + 1. \tag{21}$$

In Section I we have specialized the mean value between these two bounds for $b = 2$ and $N = 2^n$ as an *a priori* estimate for the number of shifts.

IV. Trigonometric Error

We have explained in Section I why the trigonometric elements of the matrices F_k will not always be exact. Instead of the correct matrices F_k, slightly different matrices $F_k + \Delta F_k$ will be used, and instead of the desired transform $x_n = F_n F_{n-1} \cdots F_1 x_o$ there results

$$x_n + \Delta x_n = (F_n + \Delta F_n)(F_{n-1} + \Delta F_{n-1}) \cdots (F_1 + \Delta F_1) x_o. \tag{22}$$

To first order, the trigonometric error vector is

$$\Delta x_n \simeq \sum_{k=1}^{n} F_n \cdots F_{k+1} (\Delta F_k) F_{k-1} \cdots F_1 x_o. \tag{23}$$

Its rms value is bounded by

$$\|\Delta x_n\| \leqslant \|x_n\| \sum_k (\|\Delta F_k\|/\|F_k\|). \tag{24}$$

To evaluate $\|\Delta F_k\|$, we note that each row of F_k holds p_k sine-cosine pairs, so that each row of ΔF_k holds, at most, $2p_k$ elements different from zero and, if there is no scaling shift, each of these is bounded by ϵ, the maximum absolute error in the sine and cosine values. By permutation of rows and columns, ΔF_k can be turned into block diagonal form holding N/p_k blocks of dimension $2p_k \times 2p_k$ each. It is easy to see [5, ch. III] that for such a matrix of blocks filled with ϵ's, the spectral norm is $\|\Delta F_k\| \leqslant 2p_k\epsilon$. We had seen earlier that, in the absence of scaling shifts, $\|F_k\| = \sqrt{p_k}$, so that (24) evaluates to

$$\frac{\|\Delta x_n\|}{\|x_n\|} \leqslant 2\epsilon \sum_{k=1}^{n} \sqrt{p_k}. \tag{25}$$

Scaling shifts multiply both $\|\Delta F_k\|$ and $\|F_k\|$ by the same factor which cancels in the ratio (24) so that (25) becomes our final bound on the trigonometric noise/signal ratio as summarized in Section I for $N = 2^n$.

V. Stochastic Analysis

We begin by observing that the errors arising from inexact trigonometric function values do not allow stochastic analysis. The same inexact values are used repeatedly in the computation; the errors are not statistically independent and a stochastic analysis is not appropriate.

For the arithmetic errors, the assumption of statistical independence seems more reasonable, and both the mean error and the variance are needed to arrive at the rms error. One would hope that the mean error, i.e., the bias in the Fourier transformed data, is zero so that only the variance will be needed, but this is not the case. The next two paragraphs explain our arguments.

The vector of mean errors $E(e_k)$ is obtained by taking statistical expectation values of the elements of the error vector e_k in (4). For its norm, (5) translates into

$$\|E(e_k)\| \leqslant \|F_k\| \cdot \|E(e_{k-1})\| + \|E(d_k)\|. \tag{26}$$

The mean error propagates through the computation in the same way as the maximum error, and our results remain valid for the mean error after replacement of the arithmetic error vector d_k by its expectation $E(d_k)$. At the end of the computation, bound (26) can vanish only if $E(d_k)$ is a null vector for all k.

The errors generated in multiplication and shift operations depend on whether truncation or rounding is used, and on whether negative numbers are represented in complement or sign-magnitude format, but in no case can the mean error be relied upon to vanish. Consider first the case of truncation. In complement format, truncation reduces the value of positive as well as negative numbers; the error distribution is uniform from -1 to 0 with expectation $-\frac{1}{2}$. In sign-magnitude format, truncation reduces the absolute value of a number; the error distrubtion is uniform from -1 to 0 for positive numbers, and from 0 to $+1$ for negative numbers. The expected error is $\pm\frac{1}{2}$ with the sign of the error correlated with the sign of the data. It does not appear safe to neglect this correlation in order to get zero mean error. For rounding, the expected error is generally smaller than for tuncation, yet still does not necessarily vanish. For illustration, consider the error arising from rounding after a one-bit right shift in binary arithmetic. Its distribution concentrates at three values: if the bit shifted out was a zero, there is no error. If it was a 1, the error is $\pm\frac{1}{2}$ with the sign depending on whether the number is rounded up or down. The usual rules to decide between rounding up or down are based on the sign of the number or on whether it is even or odd, and thus introduce again a correlation be-

tween errors and data. This problem arises for any even-b arithmetic.

For any arithmetic we have considered, the mean error does not exceed half the maximum error. In fact, bounding the mean error by half the maximum error is not at all pessimistic for truncating two's-complement arithmetic, which appears to be most common at present for fixed-point FFT computations. Since, generally, the mean error can amount to a sizeable fraction of the maximum error, there is little merit in calculating the variance, although our analysis method can easily be applied to such calculation. Quite generally, we feel that there is little gained at the expense of greater complexity and uncertainty in calculating probabilistic error bounds for the fixed-point fast Fourier transform.

Appendix I
Machine Error

We derive a bound on the elements of the error vector d_k that arises in the computations (3) of the kth stage. The units of d_k are those of the least significant computer digit in the elements of x_k. There are some differences between algorithms, and also for the first or last stage in a given algorithm, but we will neglect these special cases. The error in these cases is smaller than the worst case bound derived in the following but, except for very short data arrays, the overall reduction amounts to less than a factor $\frac{1}{2}$. Also, since truncation is more simply programmed and produces greater error than rounding, truncation will be assumed even though it generates a bias in complement arithmetic.

In a typical stage, there are at most $2p_k$ multiplications to compute one element of the new vector x_k from $2p_k$ elements of the old x_{k-1}. Each multiplication introduces a truncation error of, at most, one unit. If no shifting takes place, the error in the new element is bounded by $2p_k$ units.

If shifting is required, a natural program arrangement is this: upon detection of an overflow, the current "butterfly" is abandoned and the entire data array is shifted one digit, corresponding to a truncated division by b. The effect is that those elements of the array that had already been computed before the overflow was detected (and belong to x_k) are shifted after multiplication, while those still to be operated upon (and belonging to x_{k-1}), including those of the current butterfly, are shifted before multiplication. The effect of the shift is different for these two sets of elements and, since there may be several shifts per stage, we look at the general case of an element which is shifted, possibly several times, before and after entering the butterfly.

The shifts before butterfly computation introduce an error bounded by one unit into each element remaining in x_{k-1}. The weighted sum of p_k pairs of these give a new element for x_k. The weighting is by a sine and cosine multiplier for each pair, resulting in an error bounded by $|\sin\theta| + |\cos\theta| \leqslant \sqrt{2}$ from each pair. The p_k pairs thus introduce an error of, at most, $p_k\sqrt{2}$ into each new element of x_k. In addition, there is the multiplication error bounded by $2p_k$ units, giving a total error bounded by $(2+\sqrt{2})p_k$ units.

Any number of shifts after computation introduce an error of, at most, one unit, and each such shift divides the previous error by b. For a shifts after the butterfly computation, the error is thus bounded by $(2+\sqrt{2})p_k b^{-s} + 1$. This is dominated by the case of no shifts after butterfly computation, and we take $c_k = (2+\sqrt{2})p_k$.

For the case $b = 2$ and $N = 2^n$, the bound can be sharpened a bit by attention to detail. At most two shifts can occur in any one stage in this case. At worst, both shifts occur before the butterfly computation and generate an error bounded by $\frac{3}{4}$ in the shifted elements of x_{k-1}. In the butterfly, a new element of x_k is computed as the sum of a sine cosine-weighted pair of elements of x_{k-1}, plus one unmodified element of x_{k-1}. (This applies directly in some algorithms. In others, it applies on average in the sense that half the elements of x_k arise each from two weighted pairs and the remaining half arise each from two unmodified elements of x_{k-1}.) The shift error is thus magnified to $\frac{3}{4}(1 + |\sin\theta| + |\cos\theta|) \leqslant \frac{3}{4}(1+\sqrt{2})$. In addition, there is the error from truncating the two products, bounded by 1 for each product. The resulting total is $c_k = \frac{3}{4}(1+\sqrt{2}) + 2 = 3.81$. This value has been used in the summary of our results in Section I.

Appendix II
Real Input Data

Of course, it is straightforward to treat real input data by simply setting the imaginary part of x_o to zero and applying the FFT algorithm unchanged. Our error analysis does not require modification for this case. Although this method is straightforward, it suffers from redundancy. In the input array, the imaginary part is filled with zeros, and in the output array every element is duplicated in conjugate complex form. Two modifications of the FFT algorithm have been developed for real input data to permit the length N of all complex vectors in the transform to be halved when N is even. We do not consider the complications that arise with these methods when N is odd.

In the first of these modifications [12], a complex input vector x_o of length $N/2$ is formed by placing the N real input elements alternately into real and imaginary parts, so that the real part of x_o holds the even subscripted input samples and the imaginary part holds the odd subscripted ones. The complex FFT algorithm is then applied to x_o and produces x_{n-1} in $n-1$ stages corresponding to the prime factorization of $N/2$. One additional stage is then required to allow for the shifting of the odd subscripted input data. This last stage consists of a sum and difference butterfly followed by a regular radix-2 butterfly. Our error analysis is unchanged for the $n-1$ stages. For the last stage, we obtain $\|F_n\| = 2$ and $c_n \leqslant 5.41$ (for binary arithmetic $c_n \leqslant 4.56$). Equation (6) shows that the contribution of the last stage to the arithmetic error is determined by $c_n/\|F_n\|$, and this ratio is less than that for a regular $p = 2$ stage. It is thus conservative to consider this last stage as a regular $p = 2$ stage so that our error bound with N designating the length of the real input vector applies unchanged to the modified algorithm. Our trigonometric error bound is also unchanged.

The second modification [13] involves an adaptation of the algorithm such that the transform is obtained in $n-1$

stages corresponding to the prime factorization of $N/2$. Our error analysis applies unchanged to these stages. Strictly, there is a remnant of the nth stage in the form of a single sum and difference butterfly to compute the two real spectral points at zero and at maximum frequency. If the error generated in this extra butterfly is neglected, our results apply with N being only half the length of the real input data array.

References

[1] J. W. Cooper, *J. Magnetic Resonance*, vol. 22, p. 345, 1976.
[2] P. D. Welch, *IEEE Trans. Audio Electroacoust.*, vol. AU-17, p. 151, 1969; also in [9].
[3] A. V. Oppenheim and C. J. Weinstein, *Proc. IEEE*, vol. 60, p. 957, 1972.
[4] T.-Thong and B. Liu, *IEEE Trans. Acoust., Speech, Signal Processing*, vol. ASSP-24, p. 563, 1976.
[5] J. H. Wilkinson, *Rounding Errors in Algebraic Processes.* Englewood Cliffs, NJ: Prentice-Hall, 1963.
[6] M. Abramowitz and I. A. Stegun, *Handbook of Mathematical Functions*, U.S. Government Rep. NBS-AMS55, 1964.
[7] J. W. Cooper, I. S. Mackay, and G. B. Powle, *J. Magnetic Resonance*, vol. 28, p. 405, 1977.
[8] J. A. den Hollander, Rijksuniversiteit Leiden, The Netherlands, personal communication.
[9] B. Liu, Ed., "Digital filters and the fast Fourier transform," *Benchmark Papers in Electrical Engineering and Computer Science*, vol. 12. Stroudsburg, PA: Dowden, Hutchinson & Ross, 1975.
[10] F. Theilheimer, *IEEE Trans. Audio Electroacoust.*, vol. AU-17, p. 158, 1969. E. O. Brigham, *The Fast Fourier Transform.* Englewood Cliffs, NJ: Prentice-Hall, 1974.
[11] B. Noble, *Applied Linear Algebra.* Englewood Cliffs, NJ: Prentice-Hall, 1969.
[12] C. Bingham, M. D. Godfrey, and J. W. Tukey, *IEEE Trans. Audio Electroacoust.*, vol. AU-15, p. 56, 1967; J. W. Cooley, P. A. W. Lewis, and P. D. Welch, IBM Res. Paper RC-1743, 1967; *J. Sound Vib.*, vol. 12, p. 315, 1970.
[13] G. D. Bergland, *Commun. Ass. Comput. Mach.*, vol. 11, p. 703, 1968; *IEEE Trans. Audio Electroacoust.*, vol. AU-17, p. 138, 1969.

On the Problem of Designing IIR Digital Filters with Short Coefficient Word Lengths

HON-KEUNG KWAN

Abstract—**The paper presents an algorithm for designing IIR digital filters with short coefficient word lengths. The algorithm has the flexibility of obtaining a better design at the expense of investing more computational time. It was found that, by incorporating Crochiere's idea of equalizing passband and stopband statistical word lengths before applying the algorithm, we are capable of obtaining a further reduction in the overall word length when compared with that obtained by applying the algorithm alone. The principle of the algorithm and the concept of statistical word lengths equalization for further word length reduction applies to all IIR digital filters of different structures and passbands, with and without their transfer functions expressed in a closed form.**

I. Introduction

DESPITE the many advantages offered by digital filters, there are some practical limitations associated with their actual implementation. The most important limitation is caused by quantization. The three major sources of quantization errors are: 1) input quantization errors, 2) arithmetic quantization errors, and 3) coefficient quantization errors. The first two types have been investigated [1]. Due to the limit of the word length available in minicomputers and the desired small word length in special-purpose hardware systems, quantization errors associated with finite coefficient word length become critical and may lead to a filter that does not satisfy its original specifications. Since the cost and complexity of implementation depends on the coefficient word length, the word length to be chosen should be minimum but still sufficient to fulfill the desired requirements.

It is the last limitation which we have addressed ourselves to in this paper. As a result, an algorithm was formulated. It has the flexibility of yielding a better reduction in coefficient word length at the expense of investing more computational time. It was found that, by incorporating Crochiere's idea [2], [3] of equalizing passband and stopband statistical word lengths before applying the algorithm, we are capable of obtaining a better design when compared with that obtained by employing the algorithm alone. The efficiency of the algorithm was demonstrated by employing Crochiere's three elliptic low-pass cascaded digital filters [2] as examples.

Manuscript received December 29, 1977; revised October 30, 1978 and May 8, 1979.

The author is with the Department of Electrical Engineering, Imperial College of Science and Technology, London, England.

II. Problem Formulation

Consider the cascade form elliptic digital transfer function as

$$T(Z) = K \prod_{i=1}^{N} \frac{1 + b_i Z^{-1} + Z^{-2}}{1 + e_i Z^{-1} + f_i Z^{-2}} \qquad \forall w \tag{1}$$

where

0096-3518/79/1200-0620$00.75